



Marcar aula



## De olho no erro

Módulo 37

**Aprenda a identificar e corrigir os erros mais comuns e torne seu código mais eficiente e confiável.**

# 01

**Não padronizar os dados antes de aplicar o PCA.**

**Descrição:** O PCA é sensível à escala dos dados, e não padronizar os dados pode levar a resultados incorretos, pois variáveis com maior variância podem dominar os componentes principais.

**Tipo de Erro:** Erro lógico.

**Exemplo:**

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
pca.fit_transform(data)
```

**Correção:**

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

```
scaler = StandardScaler()  
data_scaled = scaler.fit_transform(data)
```



Marcar aula



**Passo 1:** Importe a classe StandardScaler do sklearn.preprocessing.

**Passo 2:** Aplique o StandardScaler para padronizar os dados antes de aplicar o PCA.

**Passo 3:** Use os dados padronizados para ajustar e transformar com o PCA.

## 02

### Selecionar um número inadequado de componentes principais.

**Descrição:** Escolher muitos ou poucos componentes principais pode resultar em perda de informação ou em um modelo complexo demais.

**Tipo de Erro:** Erro lógico.

#### Exemplo:

```
pca = PCA(n_components=10)  
pca.fit(data_scaled)
```

#### Correção:

```
pca = PCA(n_components=0.9)  
pca.fit(data_scaled)
```

**Passo 1:** Determine a variância explicada desejada (por exemplo, 90%).



Marcar aula



# 03

## Interpretar incorretamente os componentes principais.

**Descrição:** Iniciantes podem não entender que os componentes principais são combinações lineares das variáveis originais, o que pode levar a interpretações erradas dos resultados.

**Tipo de Erro:** Erro conceitual.

### Exemplo:

```
print(pca.components_)
```

### Correção:

```
import pandas as pd

components_df = pd.DataFrame(pca.components_, columns=data.columns)
print(components_df)
```

**Passo 1:** Crie um DataFrame para visualizar os componentes principais com os nomes das variáveis originais.

**Passo 2:** Analise as cargas dos componentes para entender a contribuição de cada variável.

# 04



Marcar aula



**Descrição:** Ignorar a verificação da variância explicada pode levar a uma escolha inadequada do número de componentes principais.

**Tipo de Erro:** Erro lógico.

## Exemplo:

```
pca = PCA(n_components=4)
pca.fit(data_scaled)
```

## Correção:

```
pca = PCA(n_components=4)
pca.fit(data_scaled)
print(pca.explained_variance_ratio_)
```

**Passo 1:** Após ajustar o PCA, use o atributo `explained_variance_ratio_` para verificar a variância explicada.

**Passo 2:** Analise se a variância explicada é suficiente para suas necessidades.

# 05

## Aplicar PCA em dados categóricos sem pré-processamento adequado.

**Descrição:** O PCA requer dados numéricos, e aplicar diretamente em dados

categóricos pode resultar em erros ou resultados inválidos.

**Tipo de Erro:** Erro de tipo.



Marcar aula



```
pca = PCA(n_components=2)
pca.fit_transform(categorical_data)
```

### Correção:

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder()
categorical_data_encoded = encoder.fit_transform(categorical_data).toarray()

pca = PCA(n_components=2)
pca.fit_transform(categorical_data_encoded)
```

**Passo 1:** Use OneHotEncoder para transformar dados categóricos em numéricos.

**Passo 2:** Aplique o PCA nos dados transformados.

**? Tem dúvidas sobre esta lição? Faça uma pergunta a um tutor!**

Pergunte a um tutor

**Ainda não há comentários**

Adicionar primeiro comentário

Próximo:



Marcar aula



Marcar aula como concluída

Você gostou do conteúdo desta lição?



WhatsApp

+55 (11) 4200-2991

Telefone

+55 (11) 4200-2991

Política de privacidade

Pt

Es

En